

</> Syntax & Indentation

INSTANT

📖 Variables & Data Types

🚀 Data Analysis Diploma • Session 09

PYTHON FUNDAMENTALS

Syntax · Variables · Data Types · Type Casting · Operators

4 Modules

Core Topics

Syntax • Types • Cast • Ops

16 Slides

Curriculum Depth

Python 3.12 Fundamentals

Hands-on

Code Examples

PEP 8 Standards & Logic

Start Session 09 →

🔄 Type Casting & Conversion

⚙️ Arithmetic & Logic Operators

— CURRICULUM OVERVIEW

What We'll Cover Today

Master the foundational building blocks of Python programming for data analytics:

INTERACTIVE AGENDA & NAVIGATION

01



01. Intro to Python

Syntax, Semantics, Code Structure & why Indentation matters.

02



02. Variables & Data Types

Storing, naming rules, dynamic typing, and classifying data.

03



03. Type Casting

Converting data seamlessly between int, float, str, and bool.

04



04. Operators

Arithmetic, Comparison, and Logical operators for math and logic.



PART 1

Intro to Python

Understanding Syntax, Semantics, and why Indentation matters

— SYNTAX VS SEMANTICS

Syntax vs Semantics: What's the Difference?

Distinguishing structural language rules from runtime logic and code meaning:

</> SYNTAX

Valid Error

Grammar & Structural Rules

The rules that define correctly structured programs. It's about the correct arrangement of words and symbols.

Analogy: Grammar in a sentence. "Python is fun" follows grammar rules.

```
# CORRECT SYNTAX (Colon + Indentation)
```



```
if x > 10:
    print("Big!")
```

→ Python Interpreter: Clean compilation

🌟 SEMANTICS

Valid Error

Meaning & Execution Logic

The meaning or interpretation of the code. It's about what the code does when it runs.

Analogy: The meaning of a sentence. "Python is fun" means you enjoy coding.

```
# CORRECT SEMANTICS (int + int)
```



```
x = 5 + 3
print(x)
```

→ Console Output: 8

— INDENTATION RULES

Python **Indentation**: Structure Without Braces

Unlike C++ or Java using braces {}, Python enforces 4-space indentation for clean readable code:

Unlike C++ or Java that use braces , Python uses **indentation**. All statements in a block must be indented at the same level.

✓ **Correct (4 Spaces)**

X Wrong Indent

script.py

✓ PEP 8 Valid

```
age = 18
if age >= 18:
    print("Adult")
    can_vote = True
else:
    print("Minor")
    can_vote = False
```

Tip: Most editors automatically convert Tab to 4 spaces.

>_ TERMINAL OUTPUT SIMULATOR

```
$ python script.py
```

Adult

Process finished with exit code 0

PEP 8 STANDARD: Always use 4 spaces per indentation level. Mixing tabs and spaces leads to hard-to-find bugs!

— CODE READABILITY

Comments: Making Your Code Readable

Explaining why code was written, docstrings, and industry best practices:

SINGLE-LINE COMMENT (#)

IGNORED BY PYTHON

```
# This is a single-line comment
price = 99.99 # Price in USD

# You can also comment out code:
# print("This won't run")
```

MULTI-LINE DOCSTRING (""")

DOCUMENTATION

```
"""
This is a multi-line comment.
Used for docstrings and
longer explanations.
"""
```

5 COMMENTING BEST PRACTICES

- ✓ Explain why, not what — code shows what, comments explain reasoning
- ✓ Keep comments up to date — outdated comments are worse than none
- ✓ Comment complex logic — if it took time to figure out, comment it
- ✓ Use docstrings (""") for functions and modules
- ✓ Don't state the obvious — # increment i is redundant



PART 2

Variables & Data Types

How Python stores, names, and classifies data in memory

— STORAGE BOXES

What Are **Variables**? Your Data Storage Boxes

Assigning values, multiple assignment tricks, and Python dynamic typing:

Variables are containers for storing data. In Python, you **don't declare the type** — just assign a value and Python creates the variable automatically.

ASSIGNMENT & MULTIPLE ASSIGNMENT

```
# Create variables by assigning values
name = "Alice" # String
age = 25 # Integer
pi = 3.14159 # Float
is_active = True # Boolean

# Multiple assignment & value swap
x, y, z = 10, 20, 30
x, y = y, x # Swapped!
```

DYNAMIC TYPING LAB

int str float

```
data = 42
print(type(data)) # <class 'int'>
```

KEY CONCEPT: Python variables are dynamic labels pointing to values in memory. The same variable name can hold an `int` then later a `str`!

— PEP 8 NAMING

Naming Rules: How to Name Variables Correctly

Mastering snake_case, reserved keywords, UPPER_CASE constants, and good vs bad names:

THE RULES

- Must start with a letter or underscore (_)
- Can contain letters, numbers, and underscores only
- Case-sensitive: Name and name are different
- Cannot use reserved keywords (if, for, class, etc.)
- Use snake_case for variable names

INTERACTIVE NAME VALIDATOR

Type any variable name to test Python rules:

student_count

✓ VALID VARIABLE NAME

Perfect! Valid Python snake_case variable name.

Quick Presets: user_name • 2nd_place • class

PRO TIP: Choose descriptive names that explain what the variable holds. `student_count` is better than `n`. Use UPPER_CASE for constants.

CORE TYPES

Basic Data Types: **int**, **float**, **str**, **bool**

Inspecting types with `type()`, memory efficiency, and preventing type errors:

#int

Whole numbers, positive or negative

```
x = 42
y = -7
z = 0
type(x) # <class 'int'>
```

#float

Decimal numbers with a point

```
pi = 3.14
temp = -5.5
sci = 2.5e3
type(pi) # <class 'float'>
```

#str

Text inside quotes (single or double)

```
name = "Ali"
msg = 'Hi!'
big = """OK"""
type(name) # <class 'str'>
```

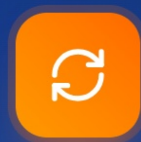
#bool

True or False values only

```
ok = True
done = False
check = 5 > 3
type(ok) # <class 'bool'>
```

WHY DATA TYPES MATTER:

- **Memory efficiency:** int uses less memory than float.
- **Correct operations:** prevents adding str to int bugs.
- **Data validation:** verifies inputs before calculating.



PART 3

Type Casting

Converting data seamlessly from one type to another

— EXPLICIT & IMPLICIT

Type Conversion: Changing Data Types on Demand

Built-in functions `int()`, `float()`, `str()`, `bool()` and auto-conversion mechanics:

Function	Converts To	Example
<code>int()</code>	Integer	<code>int("42") → 42</code>
<code>float()</code>	Float	<code>float("3.14") → 3.14</code>
<code>str()</code>	String	<code>str(100) → "100"</code>
<code>bool()</code>	Boolean	<code>bool(1) → True</code> , <code>bool("") → False</code>

INTERACTIVE CONVERSION LAB

User Input Is Always String!

Enter age (str):

```
age_str = input("Your age: ") # "25"
print(type(age_str)) # <class 'str'>

# Convert to int for math
age = int(age_str) # 25
birth_year = 2026 - age # 2001
```

IMPLICIT CONVERSION

```
# Python auto-converts int + float
result = 5 + 3.2
print(result) # 8.2
print(type(result)) # <class 'float'>

# But str + int raises TypeError!
```



PART 4

Operators

The mathematical and logical tools that manipulate your data

— ARITHMETIC ENGINE

Arithmetic Operators: Math in Python

Addition, Subtraction, Multiplication, Division, Floor Division, Modulus & Exponentiation:

INTERACTIVE MATH SIMULATOR

A =

10

B =

4

Operator	Name	Description	Live Expression Result (10, 4)
+	Addition	Adds two values	10 + 4 → 14
-	Subtraction	Subtracts right from left	10 - 4 → 6
*	Multiplication	Multiplies two values	10 * 4 → 40
/	Division	Divides, returns float	10 / 4 → 2.50
//	Floor Division	Divides, rounds down	10 // 4 → 2
%	Modulus	Returns remainder	10 % 4 → 2
**	Exponentiation	Raises to power	10 ** 4 → 10000

COMMON USE CASES:

//

 floor div for splitting items evenly

%

 modulus for even/odd: `n % 2 == 0`

COMPARISON & LOGIC

Comparison & Logical Operators: Making Decisions

Evaluating boolean expressions with ==, !=, >, <, and, or, not:

COMPARISON OPERATORS		
==	Equal to	5 == 5 → True
!=	Not equal to	5 != 3 → True
>	Greater than	7 > 4 → True
<	Less than	3 < 8 → True
>=	Greater or equal	5 >= 5 → True
<=	Less or equal	4 <= 4 → True

LOGICAL OPERATORS		
and	Both must be True	True and False → False
and	Both must be True	5>3 and 2<4 → True
or	At least one True	True or False → True
or	At least one True	5>10 or 3<7 → True
not	Reverses the value	not True → False
not	Reverses the value	not(5 == 3) → True

Interactive Student Pass Condition:

score = 85

✓ >= 60

attendance = 90

✓ >= 80

score >= 60 AND attendance >= 80

PASSED =

True

🎓 Student Passes!

INSTANT

✔ SESSION 09 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

Next Session: Python Control Flow — If statements, Loops & Functions

↺ Restart Presentation

Open Session 10: Python Control Flow →